

A Five-Layer MLOps Architecture for Connected Automated Driving

Bastian Lampe[✉], Lutz Eckstein

Abstract—The continual assurance of safety and performance of automated driving systems (ADSs) poses significant challenges. ADSs operate in complex, dynamic, open-world environments allowing a wide range of scenarios, including ones that are rare or not foreseen during initial development. While the incorporation of artificial intelligence (AI) and machine learning (ML) technology allows ADSs to learn from data gathered during operation and thus enables them to adapt over time, these approaches come with their own challenges. A key advantage of ADSs compared to human drivers is their greater ability to gather data collectively across a fleet of vehicles, or even across multiple fleets operated by different entities, and to learn from this data collectively. Vehicles can share and combine their data to identify additional learning opportunities otherwise missed by individual vehicles. This creates new opportunities to tackle the challenges of continual assurance of safety and performance, but requires the implementation of architectures that leverage the collective learning potential. Based on established MLOps principles and existing work in the field of connected automated driving, this paper presents a five-layer architecture for collective learning-enabled MLOps processes for ADSs. The goal of this architecture is to provide a conceptual blueprint for the design and implementation of MLOps processes by fleet operators and other relevant stakeholders. The paper describes the main responsibilities of each layer, their interactions, and how multi-level self-assessments enabled by the architecture can support the detection and reduction of edge cases including black swan events.

I. INTRODUCTION

The machine learning operations (MLOps) paradigm extends the DevOps [1] paradigm to the specific challenges of ML-based systems [2], [3]. When such systems are embedded in automated driving systems (ADSs), they face additional requirements due to their operation in complex, dynamic, open-world environments, where misuse, failures, or functional insufficiencies can lead to severe harm to people, property, or the environment.

To address these challenges, several regulations and standards have been established. In the EU, the AI Act [4] provides a uniform legal framework for AI systems, including high-risk AI systems such as ADSs. The AI Act requires sector-specific legislation to be aligned with its mandatory requirements, such as the Type Approval Framework Regulation (EU) 2018/858 [5], the General Safety Regulation (EU) 2019/2144 [6], and implementing rules for ADS type-approval in (EU) 2022/1426 [7].

Important standards include ISO 26262 [8] for functional safety (FuSa), ISO 21448 [9] for safety of the intended functionality (SOTIF), and ISO/TS 5083 [10] for ADS

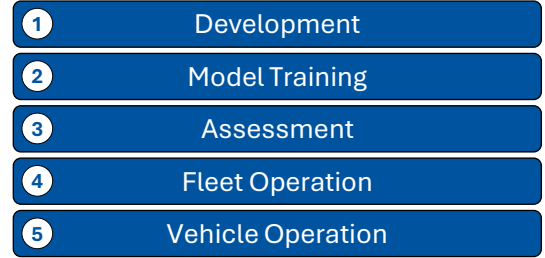


Fig. 1. The five layers of the proposed MLOps architecture.

safety design, verification, and validation. AI systems are addressed in ISO/PAS 8800 [11], which provides a framework for managing AI safety that builds upon ISO 26262 and ISO 21448. UL 4600 [12] describes assessment criteria for safety cases applicable to vehicles operating in autonomous mode and incorporating AI technology. At the UN level, the NATM Guidelines for Validating ADS [13] aim to provide a repeatable, objective and evidence-based multi-pillar approach to validating ADS safety. A draft UN Global Technical Regulation (GTR) [14] builds on this guidance by defining harmonized ADS requirements and corresponding compliance assessments. These documents provide important guidelines, however, they do not specify concrete architectures that are able to meet the requirements. While this is intentional to allow for flexibility in implementation, it also means that the question of how to design and implement MLOps processes for ADSs remains open.

Both research [3], [15], [16], [17], [18], [19] and industry [20], [21], [22], [23] have made important contributions to defining abstractions, requirements or components of MLOps processes that can be applied to ADSs. However, there remains a need for a comprehensive architecture that integrates these contributions into a coherent conceptual blueprint for the implementation of an MLOps process for ADSs.

Moreover, previous work has not yet focused on the potential of vehicle connectivity to expand the detection of learning opportunities beyond edge cases that can be detected in the data of individual vehicles. This paper explains how black swan events as defined by Shalev-Shwartz et al. [22] can be detected through collective assessments across multiple vehicles as part of an MLOps process following the presented architecture. The contributions of this paper are:

- A five-layer MLOps architecture for ADSs enabling collective, continual learning, and the continual assurance of safety and performance.
- A description of layer responsibilities and cross-layer interactions that realize the closed MLOps loop.
- A multi-level self-assessment concept for detecting and reducing edge cases including black swan events.

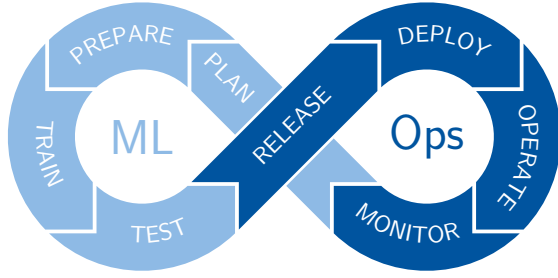


Fig. 2. The MLOps lifecycle describes the continuous process of integrating ML model development and model operations to automate and streamline the software delivery pipeline of ML models.

II. RELATED WORK

The MLOps paradigm extends the DevOps paradigm to the specific challenges of developing and deploying ML models, i.e., for the operationalization of ML products. The MLOps paradigm includes “*aspects like best practices, sets of concepts, as well as a development culture when it comes to the end-to-end conceptualization, implementation, monitoring, deployment, and scalability of machine learning products*” [3]. It needs to address ML-specific challenges described by, e.g., Sculley et al. [2]. Table I summarizes a selection of these key ML-specific challenges.

TABLE I

KEY ML-SPECIFIC CHALLENGES THAT NEED TO BE ADDRESSED BY THE MLOPS PARADIGM.

Challenge	Description
Entanglement	ML models often entangle various input signals and depend on interdependent hyperparameters, resulting in the so-called CACE principle: Changing Anything Changes Everything. A small change in any dependency can influence all others.
Data Drift	The performance of ML models is affected by data inputs that may change over time, as the external world constantly evolves. Even changes in the input signals that seem beneficial can have negative effects on the overall system performance.
Feedback Loops	Deployed models may directly or indirectly affect each other, and their own future training data, making it difficult to predict their behavior before deployment.

The MLOps lifecycle depicted in Fig. 2 is an abstract illustration of a continuous process that integrates ML model development and operations to automate and streamline the delivery of ML models into production environments. The infinity loop represents the continuous cycle that includes a set of iterative phases described in Table II.

Fig. 3 provides an architecture of a typical MLOps process that realizes the MLOps lifecycle depicted in Fig. 2. ML models are served and provide a prediction service to consumers of said service. The prediction service receives its input data from a feature store. Alternatively, the service may also consume input data directly from the service consumer.

The performance of the prediction service is monitored and if the performance degrades, e.g., due to data drift (cf. Table I), an automated training pipeline for re-

TABLE II

DESCRIPTION OF THE KEY PHASES WITHIN THE MLOPS LIFECYCLE, DIVIDED INTO ML AND OPS SECTIONS.

Section	Phase	Description
ML	Plan	Analysis of available data, formulation of requirements, aligning development goals with business objectives, and defining relevant KPIs.
	Prepare	Collecting, cleaning, and labeling data, including handling of missing or incorrect data, and ensuring data quality for training.
	Train	Using prepared data to train ML models including model selection, hyperparameter tuning, and experiment tracking.
	Test	Evaluating the performance of trained models on data not used for training using predefined metrics.
Ops	Release	Preparing ML models for deployment, including versioning, approvals, auditing, and packaging into artifact repositories.
	Deploy	Deploying packaged ML models into production environments.
	Operate	Managing the live system, ensuring performance, scalability, uptime, and infrastructure across Dev, QA, and production environments.
	Monitor	Continuously tracking model performance, identifying model drift, faults, and anomalies.

training the model can be triggered. Model training in this case includes updating model weights to increase model performance, applying the model to validation data to select a model for testing, and testing the model on dedicated testing data to give an unbiased estimate of the model performance. Training runs are tracked in a metadata store and corresponding models are saved to a model registry.

Severe performance degradations or other anomalies in the performance monitoring can also trigger data recordings that can be fed into the feature store, which includes preprocessing steps that transform the raw data into useful features.

They may also trigger a more thorough data analysis by developers. These may perform exploratory data analysis, adjust the automated training pipeline, request more or different annotated data for training, experiment with new model architectures, or adjust training hyperparameters.

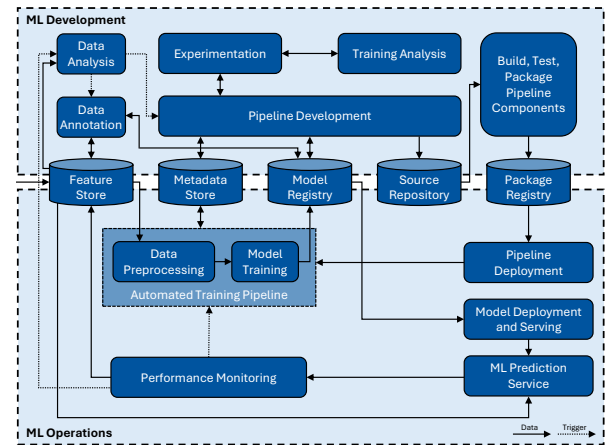


Fig. 3. Architecture of a typical MLOps process that realizes the MLOps lifecycle. The figure depicts the most important tasks and storage components, and shows the general data flow between them. The figure builds upon those depicted in [3], [24], [25], [26].

TABLE III

PROPERTIES OF ADS-EQUIPPED VEHICLES THAT REQUIRE THE DESIGN OF AN MLOPS ARCHITECTURE FOR ADSS TO DIFFER FROM MANY CONVENTIONAL MLOPS PROCESSES.

Challenge	Description
Distributed	There is not a single or few data center-based endpoints that provide a scalable prediction service to consumers; instead, each ADS-equipped vehicle comes with its own local prediction services, e.g., for object detection, behavior prediction, and behavior planning.
Isolated	There are relatively strict limitations regarding the transmission of data from the prediction service in ADS-equipped vehicles to offboard systems. Sensor data, for example, can only be shared to a very limited degree with current technology.
Self-Sufficient	ADS-equipped vehicles must be able to – at least temporarily – operate without connectivity to offboard systems, as they need to maintain safety even in the event of a loss of connectivity to these systems.
High-Risk	In the context of the AI Act, ADS-equipped vehicles can be considered high-risk systems with the potential to cause significant harm and must therefore conform to strict regulatory requirements and standards.
Heterogeneous	There may be different hardware and software configurations in different ADS-equipped vehicles, leading to variations in their capabilities and performance.
Limited	With respect to resources such as computational power, memory, and energy supply, ADS-equipped vehicles are much more limited compared to data centers.
Interacting	ADS-equipped vehicles directly influence each other’s operational context, making vehicle functions interdependent in multi-vehicle scenarios.

Both the resulting models and the training pipeline itself are analyzed with regard to their performance. Entanglement (cf. Tab. I) makes these steps particularly difficult.

Experiments are tracked in a metadata store and corresponding models are saved in a model registry. If the training pipeline performance and the model performance meet their respective requirements, the source code repository of the automated training pipeline is updated and automatically built, tested and packaged. Resulting artifacts are deployed, possibly going through a staging and quality assurance (QA) environment before.

Once an improved model generated by the automated training pipeline is pushed to the model registry, the prediction service can be updated with the new model, and the cycle starts anew. A key characteristic of this architecture is that developers do not directly produce models to be deployed but rather develop and maintain the automated training pipeline that produces the models and evaluates their performance.

III. CHALLENGES SPECIFIC TO ML-BASED ADSS

Tab. III lists challenges of operating ADS-equipped vehicles that require the design of ML systems and of MLOps architectures for ADSS to differ from many conventional, more centralized ML systems and MLOps architectures. The combination of these properties makes the design of MLOps architectures for ADSS particularly challenging compared to many other ML-based systems.

IV. SELF-ASSESSMENT AND LEARNING OPPORTUNITIES

The high-risk nature of ADS-equipped vehicles places a strong emphasis on the performance monitoring component of the MLOps process. In the context of MLOps for ADSS, a multi-level performance self-assessment (cf. Table IV) is important for the detection of *learning opportunities*, e.g., when the performance of an ML model is insufficient, and data recordings shall be triggered to allow developers to incorporate gained insights and new data into the continual learning process of affected ML models. Self-assessments are also important for maintaining the operational safety of a vehicle by, e.g., triggering a minimal risk maneuver in response to certain operating conditions [27].

The architecture of ADSS allows self-assessments to be conducted at different levels. An ADS following a microservices architecture may be composed of different services (e.g., object detection) that are part of different subsystems (e.g., perception) that in turn are part of the overall ADS. Multiple ADS-equipped vehicles form a fleet that may be supported by additional offboard systems. Connectivity between these fleet-level entities also allows collective assessments. Data can be shared across vehicles and offboard systems to be analyzed collectively, e.g., in a cloud backend.

TABLE IV

ASSESSMENT LEVELS IN FLEETS OF ADS-EQUIPPED VEHICLES AND CORRESPONDING CHALLENGES.

Assessment	Example Challenges
Service-Level Assessment	Limited context: Can leverage data from an individual service, but cannot account for information beyond the individual task of the service. <i>Example: Object detector may not have access to information about the road layout.</i> Confidence calibration: Inability to estimate confidence may correlate with inability to successfully perform the task. <i>Example: Object detector may be overconfident when encountering a rare object for which the confidence computation has not yet been calibrated.</i>
Subsystem-Level Assessment	Correlations of functional insufficiencies: Functional insufficiencies in multiple redundant components of a vehicle subsystem may be correlated, making it difficult to identify disagreements. <i>Example: A partially occluded object may be missed by all sensors of the perception subsystem, because they are all affected by a performance insufficiency.</i>
ADS-Level Assessment	Error detection delay: Errors may only become apparent after some time, potentially too late for effective or comfortable risk mitigation. <i>Example: A false negative detection is only recognized once the emergency braking system – which is working with a more rudimentary perception approach and near-field sensors – is activated.</i>
Collective-Level Assessment	Data volume: Live transmission of large amounts of data from multiple vehicles may be infeasible. <i>Example: Sensor data like point clouds may be too large to be transmitted in real-time, making it infeasible to compare the lidar sensor output of multiple vehicles.</i> Correlations of functional insufficiencies: Functional insufficiencies in multiple vehicles may still be correlated, making it difficult to identify mismatches. <i>Example: Multiple vehicles may not detect an object that was not present in the training data, i.e., an insufficiency in specification.</i>

Table IV lists different levels at which the assessment of whether data shall be recorded can be performed, and exemplary challenges that may arise at these levels.

To better understand the learning opportunities that arise in the context of a fleet of ADS-equipped vehicles, a *confidence-performance matrix* is introduced in Fig. 4. It describes different combinations of the level of confidence a system can have in meeting its current functional requirements and whether the requirements are actually met. A “system” can in this case refer to different levels, as described in Tab. IV. The matrix is inspired by the visualization of scenario categories in Fig. 6 of ISO 21448 (SOTIF) [9]. There, scenarios are categorized based on whether they are *known* or *unknown*, and whether they are *hazardous* or *not hazardous*. In ISO 21448, the knowledge dimension reflects the state of knowledge within the SOTIF lifecycle. In contrast, the confidence-performance matrix in Fig. 4 refers to the *runtime state* of the system itself.

Reliable system behavior is characterized by a high performance and high confidence. There is no immediate need to improve the system in this case. Data recordings may be triggered, but are not required. A high performance coupled with a low confidence will lead to defensive behavior. Since the system does not know whether its actual performance is sufficient, it shall employ safety functions. Data recordings can be triggered in this case, since the low confidence can be used as a trigger, and they should also be triggered, because defensive behavior may negatively affect user satisfaction, but can also have further negative consequences.

A low performance combined with low confidence may result in a violation of safety requirements. This combination represents *hazardous behavior* and can correspond to a *hazardous event*, i.e., an “event that may result in harm” [28]; therefore, safety functions shall be employed. If such an event occurs in a scenario that can lead to harm and the system is unable to control the hazardous event, such behavior may result in actual harm. Data recordings again can and should be triggered in this case.

Conversely, low performance with high confidence constitutes high-risk behavior, as no safety functions are activated to achieve a safe state. In this case, the system is unaware of its own limitations and may cause harm. The likelihood of harm can be greater compared to hazardous behavior, since safety functions are not engaged, increasing the probability that hazardous events will not be controlled, even if they could be controlled if the system was aware of them.

High-risk behavior is associated with an *unknown unknown* or *black swan* event when viewed on an individual assessment level. *Black swan* events refer to rare events in which a system fails while remaining unaware of the failure, and where the failure is not reproducible or reproducibility is unknown [22]. Although rare at vehicle level, such events can – like regular edge cases – still be numerous at fleet level. An unknown unknown on one assessment level may be transformed into a *known unknown* by moving the assessment to a higher level, e.g., by aggregating data and confidence measures from multiple redundant services (subsystem-level

	High Performance	Low Performance
High Confidence	Reliable Behavior <i>Sufficient performance with adequate confidence.</i>	High-Risk Behavior <i>System does not react to hazards, leading to increased risk.</i>
Low Confidence	Defensive Behavior <i>System employs safety functions, maintaining safety.</i>	Hazardous Behavior <i>System employs safety functions, minimizing risk.</i>

Fig. 4. The confidence-performance matrix describes different combinations of the level of confidence a system has in meeting its current functional requirements and the degree to which the requirements are actually met. The operation of the system can be classified into four categories: reliable behavior, defensive behavior, hazardous behavior, high-risk behavior. The colors indicate the associated level of risk: green (low risk) to red (high-risk). High-risk behavior corresponds to particularly important learning opportunities for the continual improvement of ADS performance.

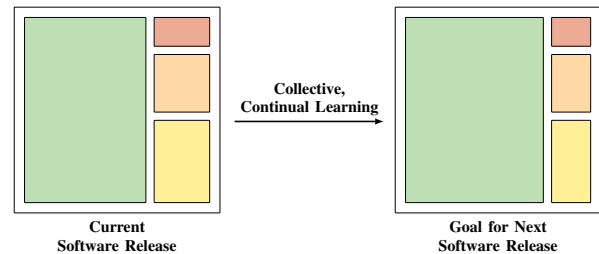


Fig. 5. The relative shares of the four behavior classes change through phases of collective, continual learning. The total colored area corresponds to the entire operational domain. The intended result is a larger share of reliable behavior (green) and smaller shares of defensive behavior (yellow), hazardous behavior (orange), and high-risk behavior (red).

assessment), or by aggregating data and confidence measures from multiple vehicles (collective-level assessment). However, there is no guarantee this is always possible.

Fig. 4 introduces *confidence* as a second dimension next to the operational system performance because, especially in high-risk systems, functional insufficiencies should be detected *before* they lead to harm, e.g., when triggering conditions leading to hazardous behavior do not lead to harm, because the scenario lacks the necessary conditions.

If an ADS-equipped vehicle enters an intersection against a red light, but there is no crossing traffic present, a data recording should still be triggered to allow the continual learning process to incorporate data of this event such that the system shows correct behavior in the future when the same or a similar scenario occurs. Preferably, a data recording is triggered even when the local perception stack is highly confident in an incorrect traffic-light interpretation. However, if the ADS uses only its own local (un-)certainty to determine

whether a data recording should be triggered, this event may be missed; hence collective assessments are required. Such collective assessments can combine observations from multiple vehicles and offboard systems to identify inconsistencies that remain invisible at the level of a single vehicle.

In reality, the class boundaries depicted in Fig. 4 are not necessarily discrete, but continuous. Nonetheless, the matrix provides a useful abstraction to describe learning opportunities that arise in fleets of ADS-equipped vehicles. The figure also illustrates the dilemma between trying to minimize the aggregation of data on higher levels due to, e.g., resource limitations or privacy concerns, and the maximization of detected learning opportunities.

Even if confidence levels in an ADS are high, there may still be a need to aggregate data and confidence measures on a higher assessment level. The need may even be highest in these cases, because the system may not be aware of its own limitations and would not employ safety functions that aim to achieve a safe state. Hence, though seemingly unintuitive, the most important learning opportunities for collective, continual learning with collective assessments may arise in scenarios where an individual ADS shows a high confidence in its current performance.

Fig. 5 illustrates a central intended result of collective, continual learning within an MLOps process, i.e., to increase the ratio of reliable behavior in the operational domain (OD) of the ADS, while decreasing the ratio of high-risk, hazardous, and defensive behavior. This may occur not only by increasing the performance of ML-based ADSs, but also through better confidence calibration.

V. FIVE-LAYER MLOPS ARCHITECTURE

In contrast to a standard MLOps architecture (cf. Fig. 3), the MLOps architecture for ADSs is structured into five layers rather than the conventional two-layer split between ML development and ML operations. Note that large parts of the functional architecture depicted in Fig. 6 would be the same in a general DevOps architecture for ADSs that includes both ML-based and non-ML-based applications.

A. Development Layer

The primary purposes of the development layer are the annotation of data for building training, validation, and testing datasets, and the development of training pipeline components for execution in the model training layer.

Specifically, this layer (1) adds and refines data annotations, potentially leveraging models from the model registry for active learning or annotation assistance; (2) performs detailed data analysis, e.g., when triggered by the assessment layer; (3) conducts experimentation; (4) analyzes training runs on development datasets, primarily to develop components for the automated training pipeline, with source code maintained in corresponding source repositories; and (5) builds, tests, and packages training pipeline components for storage in the package registry.

Similar to a typical MLOps architecture, this layer does not directly generate models for deployment, but rather

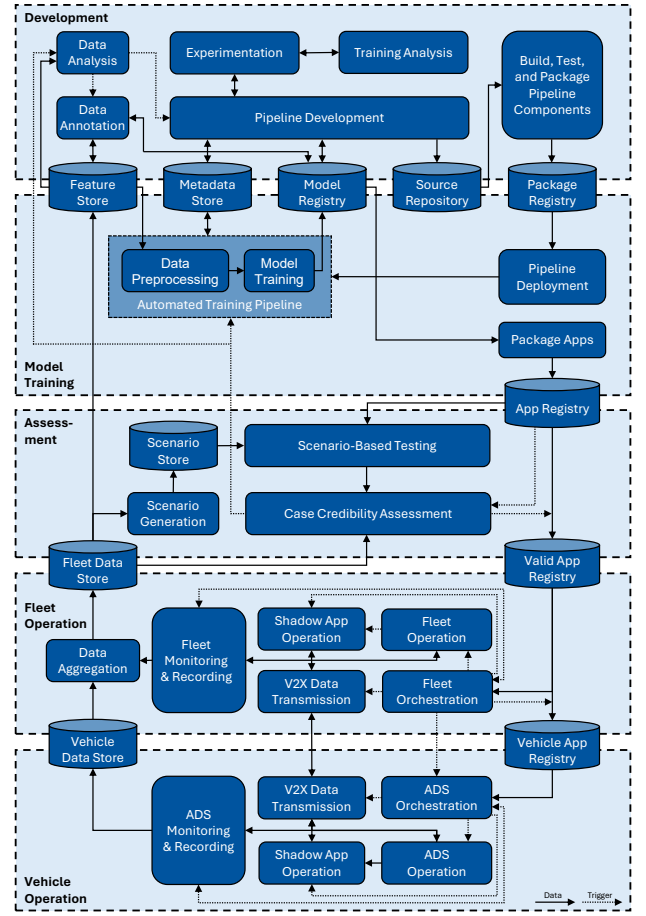


Fig. 6. Five-layer architecture of an MLOps process for fleets of ADS-equipped vehicles enabling collective, continual learning. The figure depicts the most important tasks and central storage components.

focuses on the development of training pipeline components and dataset development.

B. Model Training Layer

In contrast to the standard MLOps pipeline depicted in Fig. 3, this layer is only dedicated to the automated generation and continual improvement of applications that incorporate ML models. It does not operate prediction services; those are handled by the operations layers.

Concretely, the model training layer (1) retrieves data samples from the feature store; (2) performs data preprocessing; (3) trains and evaluates models – optionally initialized from pretrained models retrieved from the model registry – within an automated training pipeline that is (4) automatically deployed from packaged training pipeline components in the package registry. It (5) records training metadata and statistics in the metadata store; (6) registers trained models (with provenance and metrics) in the model registry; and (7) packages models into applications with explicit description of inputs and outputs, capabilities, and performance profiles.

Packaged applications are stored in the application registry and are gated by the assessment layer. The majority of aspects in the model training layer remain similar to the standard MLOps pipeline.

C. Assessment Layer

The impact of trained models on the performance of an ADS cannot be sufficiently evaluated by the model training layer alone, because ML-based applications are part of a complex system of systems producing emergent behavior that cannot be sufficiently predicted by measuring the performance of individual models. Therefore, evaluation shall also be performed in the context of the entire system of systems or a suitable model thereof. This layer primarily gates the deployment of applications to the operations layers by assessing whether the applications remain, e.g., acceptably safe.

For this purpose, this layer accesses data from the fleet data store to (1) generate scenarios for the purpose of (2) conducting scenario-based testing and to (3) continually analyze whether, e.g., a safety case for the ADS is still credible, i.e., the safety case still sufficiently argues for the absence of unreasonable risk.

Analogous to this *Safety Case Credibility Assessment* (CCA) described in Schnelle et al. [29], there should be a *User Acceptance CCA* to maintain and improve user satisfaction over the ADS lifetime. The same applies, e.g., to *security*, *privacy*, *ethical behavior*, and *environmental impact*. Each domain-specific CCA should at least assess the credibility of the domain-specific argument, and of the domain-specific evidence, as proposed by Schnelle et al. [29].

A Safety CCA shall run periodically and/or be triggered by events, e.g., when new or updated applications are available in the application registry, when measured or predicted Safety Performance Indicators (SPIs) are updated and the two no longer sufficiently align, or when significant edge cases or anomalies were detected and recorded in the fleet data store. Additional triggers may include updated regulations or standards, changes to the operational design domain (ODD), changes to the vehicle hardware, external reports of safety incidents that were not captured in the fleet data store, changes to the OD, changes to external technical systems on which the ADS relies, changes to the tools used in a CCA, and societal changes relating to the question of what is considered acceptably safe. A CCA is not limited to the results of scenario-based testing. In the context of ML systems, it shall also evaluate possible dataset insufficiencies (cf. Jeyachandran et al. [30] and ISO/PAS 8800 [11]).

Criteria to determine the absence of unreasonable risk are presented by, e.g., Favaro et al. [31]. When scenario-based testing is employed, tests may range from tests of individual services to tests involving the entire system of systems. Open-loop tests including replay/reprocessing tests and replay-to-sim tests; semi-closed-loop tests such as adaptive replay-to-sim tests [32] and advanced replay-to-sim tests [33]; and closed-loop tests can be used. Additionally, tests may include hardware-in-the-loop, software-in-the-loop, and human-in-the-loop testing.

Scenarios used for testing shall reflect already encountered real-world scenarios that can be continually extracted from the fleet data store, relevant variations of these scenarios, and other relevant scenarios that have not yet been encountered

and recorded, e.g., those sourced from expert knowledge. The fleet data store may also contain recordings of data produced by so-called shadow mode applications, which are applications that can be tested in parallel with the existing applications, but their output is not used in downstream services that are part of the causal chain affecting vehicle behavior. Additionally, the fleet data store may also contain data from which safety-related metadata can be extracted, e.g., leading SPIs such as frequency of near-misses and frequency of emergency braking, or lagging SPIs such as the frequency of collisions [12].

The Safety CCA of this layer may (4) trigger the automated training pipeline if, e.g., a retraining with new data is expected to improve the safety performance associated with the ADS. It may also (5) trigger a more detailed analysis and subsequent actions in the development layer if, e.g., the safety case is violated, for instance because the predicted item performance substantially deviates from the actual performance as measured by SPIs. Additionally, it may (6) trigger the transfer of applications from the application registry to the validated application registry if the safety case is still valid for new or updated applications as part of the overall system of systems.

The Safety CCA may also (7) revoke the validated status of applications or of the complete ADS which were previously determined acceptably safe. Additionally, (8) applications may be tagged with a label, e.g., *shadow mode* if an application is suitable for open-loop real-world testing, enabling underlying layers to deploy it in shadow mode. Additional labels may specify deployment strategies and operational guardrails (e.g., OD constraints).

Finally, (9) the Safety CCA itself should be continually updated based on, e.g., the results of the Safety CCA or new regulations and standards. This meta-analysis of the safety case shall include the analysis of assumptions, SPIs, tools and data used to analyze the validity of the safety case. This includes the analysis of the adequacy of the continuous DevOps/MLOps process itself.

For more details on ADS safety cases, refer to, e.g., the results of the VVM Project [18] or to Waymo's approach to building a safety case [21]. For more details on the continual safety assessment of ML systems, refer to [11], [12], [17], [34], [35], [36].

D. Fleet Operation Layer

The fleet operation layer supports the vehicle operation layer by (1) exchanging live data with the vehicle operation layer. This enables (2) additional supportive, cooperative, and collective services to aid vehicles in their operation. This may include providing a digital twin service that integrates data from various sources, including vehicles [37], [38]. To support this digital twin and other purposes, possibly connected, sensor-equipped roadside infrastructure may be used [39].

Not all tasks of this layer can be, or are allowed to be, fully automated. Control centers with human operators are required to oversee the operation of the fleet [7] and intervene or assist if necessary. They perform driving-related tasks

such as remote assistance, driving, or intervention, as well as non-driving-related tasks such as communication with, e.g., passengers, and emergency handling [40].

Similar to the vehicle operation layer, the fleet operation layer may (3) run shadow mode fleet applications to test new or updated software open-loop. The fleet operation layer is also responsible for (4) orchestrating the operation of different tasks and applications at fleet level [41], [42], [43], including the orchestration of tasks across multiple vehicles or other entities in the C-ITS. This also involves data recordings triggered on the collective level.

If applications are tagged for special deployment in the assessment layer, the fleet operation layer is responsible for orchestrating corresponding deployment strategies. These strategies may include *A/B testing* [44], i.e., deploying two versions of an application to different parts of the fleet and comparing their performance, *canary testing*, i.e., deploying a new version of an application to a small subset of the fleet to, e.g., limit the impact of potential issues, *rolling updates*, i.e., incrementally making updates available to a larger subset of the fleet while monitoring for issues before proceeding further, *OD-limited testing*, i.e., allowing fleet and vehicle orchestration layers to deploy applications only in specific operational domains, *controlled deployment testing*, i.e., allowing fleet and vehicle orchestration layers to deploy applications only on specific vehicles used by the fleet operator for, e.g., test track testing or field-testing with a safety driver.

Fleet monitoring (5) is another central task that can trigger data recordings and/or fleet orchestration tasks in the event of, e.g., detected anomalies, edge cases, and functional insufficiencies. Further, the fleet operation layer is responsible for (6) aggregating data recordings of fleet and vehicle operation layers, possibly enriching the data with additional information, curating data, deriving statistics, and storing data in the fleet data store. Finally, it is responsible for (7) triggering OTA updates that move applications from the validated applications registry to vehicle application registries when new or updated validated applications are available and vehicles are eligible and capable of receiving them.

E. Vehicle Operation Layer

The vehicle operation layer is responsible for operating individual vehicles. It (1) orchestrates applications (cf. Kampmann [45]) available in the vehicle application registry as currently required, (2) runs the applications that form the vehicle-based ADS responsible for the dynamic driving task (DDT), including possible safety functions implementing, e.g., minimal risk maneuvers (cf. Ackermann [27]), interactions with remote operators, interactions with passengers via internal HMI, and interactions with the environment via external HMI; it (3) runs shadow mode vehicle applications, (4) exchanges live data with the fleet operation layer, (5) monitors vehicle-based data to record relevant events, especially in case of anomalies, edge cases, ODD boundary approaches/exits, detected functional insufficiencies, and activations/re-initializations and deactivations of the ADS;

and triggers the orchestrator when a reconfiguration of the ADS is required. Finally, it (6) stores recordings in the local vehicle data store for later transmission to the fleet data store via the fleet operation layer.

Fig. 6 depicts the MLOps architecture for a single fleet of vehicles. It assumes that data can be shared between all layers for the purpose of improving and assuring the overall system performance, i.e., the organization operating the MLOps process holds all necessary access rights to the data. In a multi-fleet case, where multiple organizations operate their own fleets but nonetheless share some components of some of the layers, or in a single-fleet case where different layers – or parts thereof – are operated by different organizations, the data flow would be more restricted. Federated learning approaches could be applied in these cases.

VI. CONCLUSION

The presented five-layer architecture of an MLOps process for ADSs provides a conceptual blueprint for fleet operators and other stakeholders to implement such a process, enabling them to continually assure the safety and performance of their fleet of ADS-equipped vehicles. The architecture aims to adapt standard MLOps architectures to the specific requirements and challenges associated with ADSs, and to leverage the possibilities of collective assessments to detect and reduce black swan events, other functional insufficiencies, and failures.

By enabling collective, continual learning for ML-based ADSs, the MLOps architecture aims to increase the ratio of reliable behavior in the operational domain of an ADS over time, while decreasing the ratio of high-risk, hazardous, and defensive behavior.

The detailed descriptions of each layer's tasks and responsibilities, and of cross-layer interactions, provide a comprehensive starting point for implementing a corresponding MLOps process. Future work will include such an implementation to empirically evaluate the architecture and to identify potential gaps and areas for further improvement.

Additionally, a more detailed mapping of architecture components to the extensive requirements of regulations and standards is necessary to evaluate whether the architecture needs to be adapted or extended to meet all requirements. At the same time, further research is required to determine whether current regulations and standards should be extended to better support collective, continual learning for ML-based ADSs.

Finally, future research should investigate the applicability of the presented architecture to other classes of complex, high-risk physical AI systems.

ACKNOWLEDGEMENTS

This work was funded by the European Union under the Horizon Europe programme in the AIGGREGATE project (Grant Agreement No. 101202457). Furthermore, this research received funding from the German Federal Ministry of Research, Technology and Space within the "autotech.agil" project (FKZ 01IS22088A).

REFERENCES

- [1] ISO - International Organization for Standardization, *ISO/IEC/IEEE 32675 (Information technology — DevOps — Building reliable and secure systems including application build, package and deployment)*, Std., 2022.
- [2] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems*, vol. 28. Curran Associates, Inc., 2015.
- [3] D. Kreuzberger, N. Kühl, and S. Hirschl, “Machine learning operations (mlops): Overview, definition, and architecture,” *IEEE Access*, vol. 11, pp. 31 866–31 879, 2023.
- [4] European Parliament, Council of the European Union, “Regulation (EU) 2024/1689 (Artificial Intelligence Act),” Official Journal of the European Union, L series, 2024/1689, 12.7.2024, 2024. [Online]. Available: <http://data.europa.eu/eli/reg/2024/1689/oj>
- [5] —, “Regulation (EU) 2018/858,” Official Journal of the European Union, L series, 2018/858, 14.6.2018, 2018. [Online]. Available: <http://data.europa.eu/eli/reg/2018/858/2024-07-01>
- [6] —, “Regulation (EU) 2019/2144,” Official Journal of the European Union, L series, 2019/2144, 16.12.2019, 2019. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2019/2144/oj>
- [7] European Commission, Directorate-General for Internal Market, Industry, Entrepreneurship and SMEs, “Regulation (EU) 2022/1426,” Official Journal of the European Union L 325/1, 2022. [Online]. Available: http://data.europa.eu/eli/reg_impl/2022/1426/oj
- [8] ISO - International Organization for Standardization, *ISO 26262 (Road vehicles – Functional safety)*, Std., 2018.
- [9] —, *ISO 21448 (Road vehicles — Safety of the intended functionality)*, Std., 2022.
- [10] —, *ISO/TS 5083 (Road vehicles — Safety for automated driving systems — Design, verification and validation)*, Std., 2025.
- [11] —, *ISO/PAS 8800 (Road vehicles — Safety and artificial intelligence)*, Std., 2024.
- [12] Underwriters Laboratories, *Standard for Safety — Evaluation of Autonomous Products*, Underwriters Laboratories Std., Mar. 2023.
- [13] Working Party on Automated/Autonomous and Connected Vehicles, “New Assessment/Test Method for Automated Driving (NATM) Guidelines for Validating Automated Driving System (ADS),” June 2023.
- [14] Informal Working Group on Automated Driving Systems, “Proposal for a new United Nations Global Technical Regulation on Automated Driving Systems (ADS),” November 2025.
- [15] R. Johansson and P. Koopman, “Continuous Learning Approach to Safety Engineering,” in *CARS - Critical Automotive applications: Robustness & Safety*, Zaragoza, Spain, Sep. 2022. [Online]. Available: <https://hal.science/hal-03782627>
- [16] L. Ullrich, M. Buchholz, K. Dietmayer, and K. Graichen, “Expanding the classical v-model for the development of complex systems incorporating ai,” *IEEE Transactions on Intelligent Vehicles*, 2024.
- [17] M. Wagner and C. Carlan, “The open autonomy safety case framework,” 2024.
- [18] P. V. Consortium, “Pegasus vvm - final quality criteria and final methodology,” Jul. 2025.
- [19] D. Bogdoll, R. P. Ananta, A. Giridharan, I. Moore, G. Stevens, and H. X. Liu, “Mcity data engine: Iterative model improvement through open-vocabulary data selection,” 2025.
- [20] N. Webb, D. Smith, C. Ludwick, T. Victor, Q. Hommes, F. Favaro, G. Ivanov, and T. Daniel, “Waymo’s safety methodologies and safety readiness determinations,” 2020.
- [21] F. Favaro, L. Fraade-Blanan, S. Schnelle, T. Victor, M. Peña, J. Engstrom, J. Scanlon, K. Kusano, and D. Smith, “Building a credible case for safety: Waymo’s approach for the determination of absence of unreasonable risk,” 2023.
- [22] S. Shalev-Shwartz, M. Molnar, I. Granot, A. Shany, and A. Shashua, “A safety architecture for self-driving systems,” Mobileye, Tech. Rep., 2024. [Online]. Available: https://static.mobileye.com/website/us/corporate/files/SDS_Safety_Architecture.pdf
- [23] F. Favaro, S. Schnelle, L. Fraade-Blanan, T. Victor, M. Peña, N. Webb, H. Broce, C. Paterson, and D. Smith, “Determining absence of unreasonable risk: Approval guidelines for an automated driving system deployment,” 2025.
- [24] Google, “Mlops: Continuous delivery and automation pipelines in machine learning,” 2024, [Accessed 16-02-2026]. [Online]. Available: <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>
- [25] MLOps, “Mlops principles,” 2024, [Accessed 16-02-2026]. [Online]. Available: <https://ml-ops.org/content/mlops-principles>
- [26] A. Griciūnas, “Swirlai newsletters,” 2025, accessed: 16-02-2026. [Online]. Available: <https://www.newsletter.swirlai.com/>
- [27] S. M. Ackermann, “Safe halt as fail-safe concept for automated driving systems,” Ph.D. dissertation, Technische Universität Darmstadt, Darmstadt, 2023.
- [28] IEC - International Electrotechnical Commission, *IEC 61508 (Functional safety of electrical/electronic/programmable electronic safety-related systems)*, Std., 2010.
- [29] S. Schnelle, F. Favaro, L. Fraade-Blanan, D. Wichner, H. Broce, and J. Miranda, “Assessing a safety case: Bottom-up guidance for claims and evidence evaluation,” 2025.
- [30] J. Jeyachandran, S. Khastgir, X. Zhao, E. Barbier, and P. A. Jennings, “Introducing oasis : Odd-based ai safety in autonomous systems,” in *The IEEE International Conference on Intelligent Transportation Systems (ITSC)*, 2025.
- [31] F. Favaro, S. Schnelle, L. Fraade-Blanan, T. Victor, M. Peña, N. Webb, H. Broce, C. Paterson, and D. Smith, “Determining absence of unreasonable risk: Approval guidelines for an automated driving system deployment,” 2025.
- [32] N. Weber, “Adaptive replay-to-sim approach for developing and testing automated driving systems in urban areas,” Ph.D. dissertation, Technische Universität Darmstadt, Darmstadt, Juni 2025.
- [33] C. Glasmacher, M. Schuldes, P. Topalakatti, P. Hristov, H. Weber, and L. Eckstein, “Deliverable d13: Scenario-based model of the odd through scenario databases,” VVM Project, Tech. Rep., 2023.
- [34] P. Schleiss, F. Carella, and I. Kurzidem, “Towards continuous safety assurance for autonomous systems,” in *2022 6th International Conference on System Reliability and Safety (ICSRS)*, 2022, pp. 457–462.
- [35] S. Burton and B. Herd, “Addressing uncertainty in the safety assurance of machine-learning,” *Frontiers in Computer Science*, vol. 5, 2023. [Online]. Available: <https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2023.1132580>
- [36] D. Ratiu, T. Rohlinger, T. Stolte, and S. Wagner, “Towards an argument pattern for the use of safety performance indicators,” in *Computer Safety, Reliability, and Security. SAFECOMP 2024 Workshops*, A. Ceccarelli, M. Trapp, A. Bondavalli, E. Schoitsch, B. Gallina, and F. Bitsch, Eds. Springer Nature Switzerland, 2024, pp. 160–172.
- [37] R. v. Kempen, L. Adrian Heidrich, B. Lampe, T. Woopen, and L. Eckstein, “Combined registration and fusion of evidential occupancy grid maps for live digital twins of traffic,” in *2023 IEEE Intelligent Vehicles Symposium (IV)*, 2023.
- [38] M. S. Irfan, S. Dasgupta, and M. Rahman, “Toward transportation digital twin systems for traffic safety and mobility: A review,” *IEEE Internet of Things Journal*, vol. 11, no. 14, pp. 24 581–24 603, 2024.
- [39] C. Creß, Z. Bing, and A. C. Knoll, “Intelligent transportation systems using roadside infrastructure: A literature survey,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 7, pp. 6309–6327, 2024.
- [40] M.-M. Wolf, N. Krauss, A. Schmidt, and F. Diermeyer, “Control center framework for teleoperation support of automated vehicles on public roads,” in *2025 IEEE Intelligent Vehicles Symposium (IV)*, 2025.
- [41] B. Lampe, L. Reiher, L. Zanger, T. Woopen, R. van Kempen, and L. Eckstein, “Robotkub: Orchestrating large-scale cooperative multi-robot systems with kubernetes and ros,” in *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)*, 2023, pp. 2719–2725.
- [42] L. Zanger, B. Lampe, L. Reiher, and L. Eckstein, “Application management in c-its: Orchestrating demand-driven deployments and reconfigurations,” 2025.
- [43] L. Reiher, B. Lampe, L. Zanger, and L. Eckstein, “Event detection in c-its: Classification, use cases, and reference implementation,” Mar. 2025. [Online]. Available: <https://doi.org/10.5281/zenodo.17141343>
- [44] D. I. Mattos, J. Bosch, H. H. Olsson, A. Maryam Korshani, and J. Lantz, “Automotive a/b testing: Challenges and lessons learned from practice,” in *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, 2020, pp. 101–109.
- [45] A. Kampmann, “A dynamic service-oriented software architecture for the automotive domain,” Dissertation, RWTH Aachen University, Aachen, 2023.